M1SOL041 : Représentation et traitement des documents électroniques
Cours 5 : Transformer les documents XML avec XLST

Marine Tiger
marine.tiger@sorbonne-universite.fr

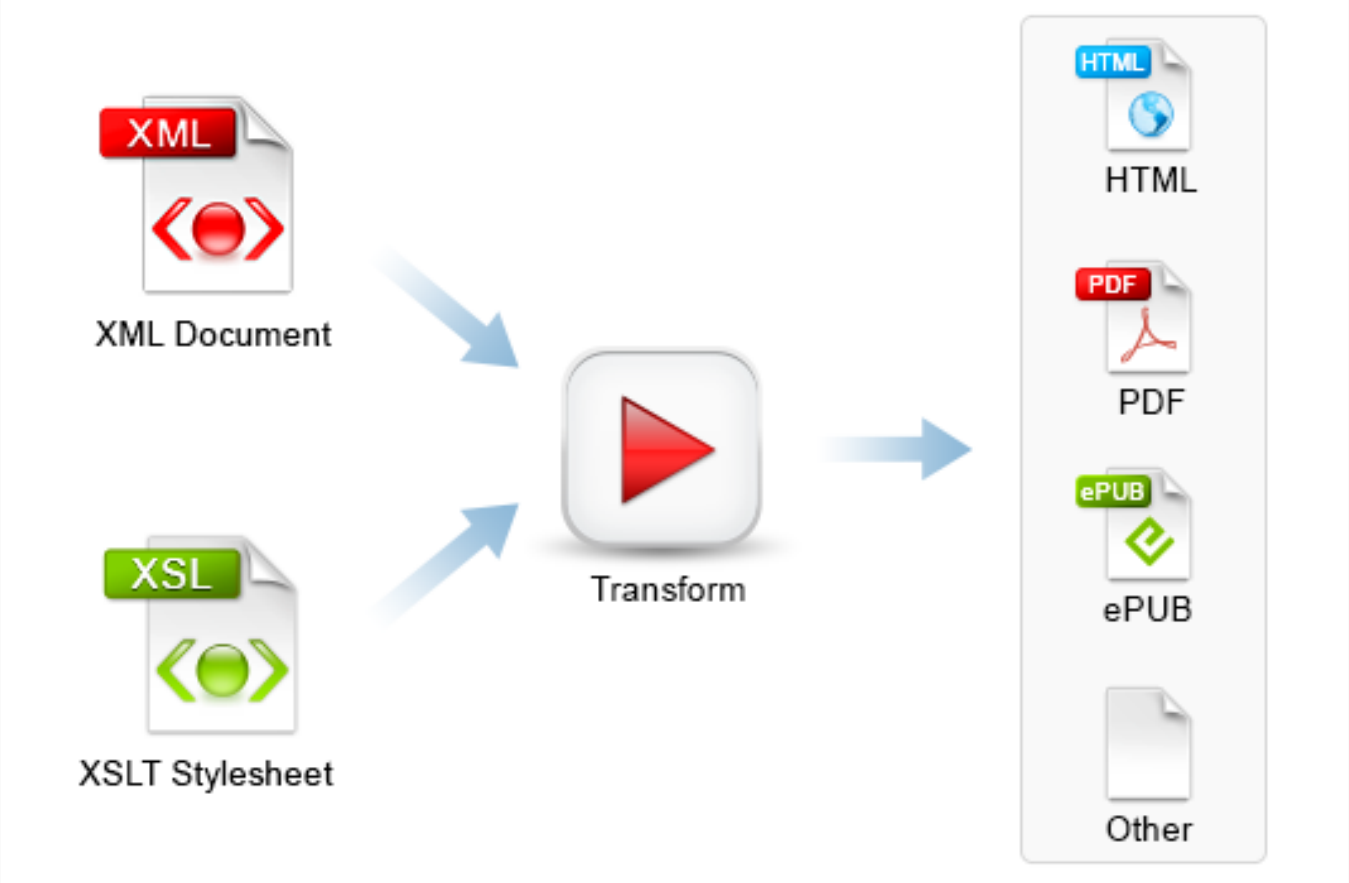
Littératures françaises et comparée ED19, CELLF, Sorbonne Université

Correction DM XPath

XSLT

Définition

Langage de transformation pour les documents XML



Utiliser XSLT dans Oxygen

Créer un fichier XSLT

- Ouvrir le fichier `CM5_DuBellay.xml`
- Ouvrir un nouveau fichier xsl (taper dans le filtre 'xsl' puis cliquer sur 'XSLT Stylesheet')

En-tête xslt

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
    xmlns:xs="http://www.w3.org/2001/XMLSchema"
    xmlns:math="http://www.w3.org/2005/xpath-functions/math"
    xmlns:tei="http://www.tei-c.org/ns/1.0"
    xpath-default-namespace="http://www.tei-c.org/ns/1.0"
    exclude-result-prefixes="xs␣math␣tei"
    version="3.0">
```

Sortie xslt

`<xsl:output>` : définie le format de sortie de la transformation XSLT. Se met après l'en-tête XSLT.

```
<!-- Sortie XML -->
    <xsl:output method="xml" indent="yes" encoding="UTF-8"/>

<!-- Sortie HTML-->
    <xsl:output method="html" indent="yes" encoding="UTF-8"/>

<!-- Sortie texte-->
    <xsl:output method="text" indent="yes" encoding="UTF-8"/>
```

Utiliser XSLT dans Oxygen

Tester des règles et voir ce qui se passe :

```
<!-- Premier test-->
    <xsl:template match="/">
        <xsl:apply-templates/>
    </xsl:template>

<!-- Deuxième test-->
    <xsl:template match="text()"/>

<!-- Troisième test-->
<xsl:template match="/">
        <xsl:copy-of select="."/>
    <xsl:template/>
```

Les règles de base de XSL

`<xsl:template>`

- Permet de donner les instructions xsl
- Utilise l'attribut `@match` pour sélectionner un élément de larbre XML
  Exemple : `<xsl:template match="TOTO">`
- On peut ajouter du texte :

```
<xsl:template match="TOTO">
        Voilà mon élément TOTO
</xsl:template>
```

Créer un élément

Méthode 1 :

```
<xsl:template match="element">
    <p> toto </p>
</xsl:template>
```

Méthode 2 :

```
<xsl:template match="element">
    <xsl:element name="nouveau_element">
        tata
    </xsl:element>
</xsl:template>
```

Créer un attribut

Méthode 1 :

```
<xsl:template match="element">
    <p type="attribut"> tutu </p>
</xsl:template>
```

Méthode 2 :

```
<xsl:template match="element">
    <xsl:element name="nouveau_element">
        <xsl:attribute name="nouvel_attribut">
            <xsl:text>valeur_attribut</xsl:text>
        </xsl:attribute>
        <xsl:text>Ici, il y avait mon élément</xsl:text>
    </xsl:element>
</xsl:template>
```

Apply-templates

`<xsl:apply-templates>` : applique le template désigné à tous les éléments ou les enfants de l'élément sélectionné.

Exemple :

```
<!-- On crée un template -->
    <xsl:template match="head">
        Title: <span style="color:#ff0000">
    <xsl:value-of select="."/></span>
    <br />
    </xsl:template>
<!-- On applique le template -->
    <xsl:template match="text">
        <p> <xsl:apply-templates select="head"/> </p>
    </xsl:template>
```

Les différentes règles en XSLT

copy et copy-of

<xsl:copy>

- Copie que le noeud courant

```
<xsl:template match="body">
<xsl :copy/>
</xsl :template>
```

<xsl:copy-of>

- Copie le noeud courant et ses enfants :

```
<xsl:template match="body">
<xsl :copy-of select="lg"/>
</xsl :template>
```

value-of

<xsl:value-of>

- Permet d'extraire la valeur d'un élément.

```
<xsl:template match="lg">
    <xsl:value-of select="."/>
</xsl:template>
```

number

<xsl:number>

- Permet de produire un nombre dans l'arbre de résultat.
- Possède plusieurs attributs :
  - @count : spécifier les noeuds à compter ;
  - @level : spécifier le nombre de niveaux
    - any : compte le nombre de noeuds.
    - single : compte le nombre de noeuds descendants d'un même parent
    - multiple : compte tous les noeuds descendants mais aussi les "siblings" et leur descendant.
  - @format : spécifier le format de numérotation.

```
<xsl:template match="mon_element_xml">
    <xsl:number count="element_a_compter" level="" format=""/>
</xsl:template>
```

Exercice

https://pad.libreon.fr/p9wWZK-ETAu6XIyAmOGjGg

Conditions et boucles

xsl :if

`<xsl:if test="chemin_xpath">`

- Une condition qui applique les règles si `@test` est vraie.

```
<xsl:template match="body">
    <xsl:if test="lg[@type='sonnet']">
        <xsl:copy-of select="."/>
    </xsl:if>
</xsl:template>
```

xsl :choose, xsl :when, xsl :otherwise

<xsl:choose>

- Est utilisé avec au moins un élément **<xsl:when>**, et en option un élément **<xsl:otherwise>** pour exprimer plusieurs conditions / sélectionner une possibilité dans une liste de choix

```
<xsl:template match="l">
    <xsl:choose>
        <xsl:when test="placeName">
           <xsl:text>LIEU</xsl:text>
        </xsl:when>
        <xsl:otherwise>
            <xsl:text>PAS DE LIEU</xsl:text>
        </xsl:otherwise>
    </xsl:choose>
</xsl:template>
```

xsl :for-each, xsl :sort, xsl :for-each-group

<xsl:for-each>

- Itère sur les noeuds sélectionnés par l'@select et applique les règles à chacun des éléments du noeud.

```
<xsl:template match="l">
    <xsl:for-each select="placeName">
        <xsl:text> TOTO </xsl:text>
    </xsl:for-each>
</xsl:template>
```

xsl :for-each, xsl :sort, xsl :for-each-group

<xsl:sort>

- Permet de changer l'ordre des noeuds avec les attributs suivants :
  - @select : objet du tri
  - @data-type : par défaut est alphabétique, mais on peut mettre "number"
  - @order : par défaut "ascending"
  - @case-order : indique si ce sont les minuscules ou les majuscules qui apparaîtront en premier.

```
<xsl:template match="element">
    <xsl:for-each select="souselements">
        <xsl:sort select="tri" order='ascending|descending'/>
        <xsl:copy-of select="."/>
    </xsl:for-each>
</xsl:template>
```

xsl :for-each, xsl :sort, xsl :for-each-group

`<xsl:for-each-group>`

- Itère sur les groupes de noeuds sélectionnés par `@select` et applique la règle à chacun d'entre eux
- `@group-by` permet de rassembler les noeuds sélectionnés en sous-groupes.

xsl :variable

```
<xsl:variable name="nom_variable" select="'valeur_variable'"/>
```

- Créer une variable dans la feuille de style.
- Utile pour la mise en forme par exemple.

```
<xsl:variable name="header">
  <font color="#9acd32"> Title </font>
</xsl:variable>

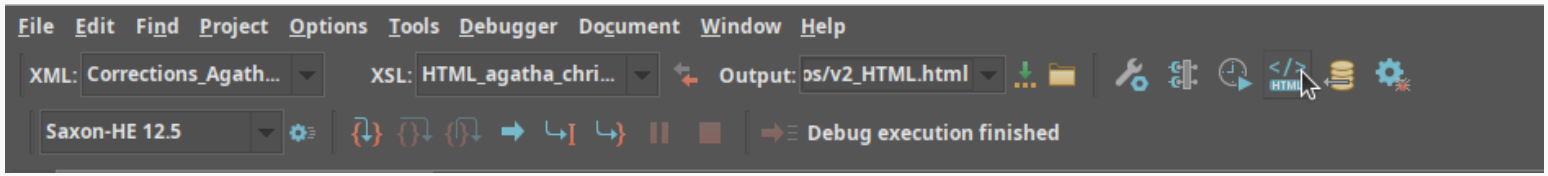
<xsl:template match="/">
  <html>
  <body>
    <xsl:copy-of select="$header"/>
  </body>
  </html>
</xsl:template>
```

Transformation HTML

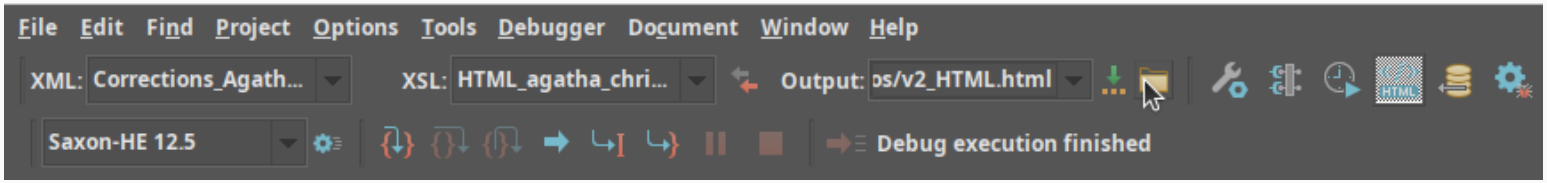
Dans Oxygen : output

```
<xsl:output method="html" indent="yes" encoding="UTF-8"/>
```

Dans Oxygen : visualiser le rendu HTML



Dans Oxygen : sauvegarder le fichier



Exercice

Faire l'exercice 3